ZK Rust Verification with Hax

Milestone 1: Field Arithmetic Ethereum

Delivered: February 4th, 2026

Prepared for Ethereum Foundation by

runtime verification

Table of Contents

- Executive Summary
  - Milestone 1 overview
  - Goals
  - Outcome Overview
  - Ongoing work location
- Target Identification
  - Mersenne31 Prime Field Arithmetic
- Hax Exploration
  - *Veredictum*: choosing Model-based verification
  - Building our Model
    - Hax Adaptations
- Reproducibility
- Connection with Specification Libraries
- Deliverables
  - Code
  - Issues
  - Pull Requests
- Next Steps

Executive Summary

The overall aim of this project is to verify real-world Rust code used in ZKVMs using Hax's Lean 4 backend.

The Lean 4 backend is currently in active development and this project seeks to provide as much feedback as possible to its development while carrying out verification efforts on production-ready Rust. Ideally producing end-to-end verification results with their corresponding pipelines and refined frameworks.

Milestone 1 is the first step towards achieving those goals. It is focused on the reconnaissance and learning of the target-space and the tooling available.

Milestone 1 overview

Milestone 1 sets the foundational groundwork needed to carry on the rest of the project. As such, this milestone's focus is on discovery, adaptability and learning of the verification targets and how the chosen tools adapt to the challenge.

Goals

Milestone 1 is guided by the following goals:

- **Target identification**: Survey zkEVM Rust repos overlapping ArkLib and ComPoly to identify suitable verification candidates.
- **Hax exploration**: Perform initial extractions with Hax on the selected candidates, recording the translation results, gaps, and providing feedback whenever possible.
- **Reproducibility**: Set up reproducible environment/pipelines/scripts to automate and replicate the desired extraction process.
- **Connection with specification libraries**: Do an initial exploration on linkage and compatibility of the extracted Lean 4 code via Hax to ArkLib and ComPoly, noting frictions and alignment issues.

Outcome Overview

The main outcome (discussed in more detail in the Deliverables section) of this milestone can be summarized in the following items:

- Establish familiarity with the Hax tool.
- Survey the ZKEVM landscape and successfully locate good verification targets.
- Research among the different verification strategies and start developing the most promising one.
- Start understanding Hax's strengths and weaknesses w.r.t. the chosen verification target and adapt to them.
- Provide feedback to Hax's developers as we used the tool.

Ongoing work location

The work discussed in this report can be found at runtimeverification/mersenne31.

Target Identification

The first step for this milestone and for the overall project is to identify a suitable target for our verification efforts.

Several items have to be taken into account as criteria to find a verification candidate:

- **Relevant component of a ZKVM**: We want our work to have a useful potential impact, so the chosen target needs to be of relative importance in a production-ready ZKVM stack.
- **Friendly-enough Rust for the Hax tool**: Although this is the exploration phase, we should be looking for code that doesn't employ many Rust fanciness, assuming that a more elemental subset of Rust will lead to less frictions down the road.
- **Relatively self contained**: A realistic end-to-end verification of a real-world Rust component of a ZKVM will need to not have huge dependencies, since these will eventually need to be brought in in some capacity or another to the verification pipeline.
- **Good overlap with specification libraries**: While our verification target is to be chosen, our Lean 4 specification targets are well defined. We will be using ArkLib and ComPoly. Hence, there should be an alignment on what these libraries cover (or can cover) and the mathematics implemented by our verification candidate.

With these in mind we surveyed two ZK frameworks, Jolt and Plonky3. Our verification target was set to be Plonky3's `Mersenne31` prime field arithmetic, which can be found in this file.

Mersenne31 Prime Field Arithmetic

Let's see how this target fits our above-defined criteria:

- Prime field arithmetic is an essential primitive in ZK frameworks. Hence, building a verification pipeline around the Mersenne31 prime field will exercise Hax against an essential ZK building block that should be part of any larger verification effort.

- Inspecting the Mersenne31 code doesn't set up a lot of alarms in terms of prohibitive Rust. While we are still in the early exploratory phase, there are not many concerns at the moment with the at hand code.
- As a primitive building block, field arithmetic doesn't have a lot of dependencies, and the mathematics of it are sufficiently self-contained. It is thus enough isolation from the rest of the ZK framework to not result in a quick dependency overwhelming.
- Our specification libraries of choice should be able to supply us with enough material to carry on the verification of field arithmetic. While it might not be the most fleshed-out part of these libraries yet, it does present a good investigative effort to highlight and possibly contribute any missing component.

Hax Exploration

Once we've identified our initial verification target, it is time to try Hax on it!

We considered two approaches: partial-extraction verification and model-based verification.

- **Partial extraction verification**: This approach consists of taking the target and perform several rounds of extraction, incrementally removing problematic items be it with Hax's `-i` flag or by commenting-out/modifying the original code. We can think of this approach as a top-to-bottom approach where we attempt to extract the entire target at first. We then remove problematic parts of it until we are left with a suitable subset that Hax is happy with.
- **Model-based verification**: In contrast with the above method, we start with a simple Rust model of the verification target, incrementally adding more and more functionality adapted to suit Hax's idiosyncrasies. Ideally we would end up with the same code as the original target. In the case of that not happening, a second-best solution would be to provide a comprehensive test suite showing behavioral equivalence between the model and the target.

Veredictum: choosing Model-based verification

Our first efforts went to the Partial extraction approach. This helped with getting familiar with Hax and to start getting a more accurate feeling on the breakages that might stem from the target code. After several attempts to modify the original verification target we concluded that the modification overhead was too great to both gauge accurately Hax's capabilities and produce any sort of meaningful end-to-end verification.

We hence turn to model-based verification for the remaining of this milestone, and potentially the project. Ideally, once a similar enough model is built, it is not too difficult to plug in the original code.

Building our Model

Similarly as when we chose our verification target, when building an incremental model we have to start at a reasonable place. After inspecting the `mersenne31.rs` file we chose to extract the `Mersenne31` implementation, which consists of the following code (removing annotations and comments):

```
impl Mersenne31 {

    pub const fn new(value: u32) -> Self {
        Self { value: value % P }
    }

    pub(crate) const fn new_reduced(value: u32) -> Self {
        debug_assert!((value >> 31) == 0);
        Self { value }
    }

    pub const fn new_checked(value: u32) -> Option<Self> {
        if (value >> 31) == 0 {
            Some(Self { value })
        } else {
            None
        }
    }

    pub const fn new_array<const N: usize>(input: [u32; N]) -> [Self; N] {
        let mut output = [Self::ZERO; N];
        let mut i = 0;
        while i < N {
            output[i].value = input[i] % P;
            i += 1;
        }
        output
    }

    pub const EXT_TWO_ADIC_GENERATORS: [[Self; 2]; 33] = [
        [Self::ONE, Self::ZERO],
        [Self::new(2_147_483_646), Self::new(0)],
```

```
        ...
    ];
}
```

However, there are a number of traits in `field.rs` that need to be extracted and implemented. We built our model leaving out some of the traits originally needed. The traits that we ended up adapting from `field.rs` are the following:

- `PrimeCharacteristicRing`
- `PrimeField32`
- `PrimeField64`

  We are also using the following traits:
- `Sized`
- `Default`
- `Clone`
- `Add`
- `AddAssign`
- `Sub`
- `SubAssign`
- `Mul`
- `MulAssign`
- `Neg`
- `Sum`
- `Product`

The following are some of the adaptations and limitations that we found when extracting the traits from our `field.rs` and implementing them in our `mersenne31.rs` model.

Hax Adaptations

The following is a description of Rust changes we had to make to the targeted code to obtain better Lean 4 generated code.

For a more detailed diff between the original Rust and our models we recommend looking at the annotations in our models.

Mutual Trait Dependency

There is a decent amount of mutual dependency in the traits of `field.rs`. While this is not problematic in Rust, mutually dependent definitions are not supported in Hax yet.

As an example of how this mutual dependency shows in `field.rs`, consider the definition of trait `PrimeCharacteristicRing`:

```
pub trait PrimeCharacteristicRing:
    Sized
    + ...
{
    /// The field `ℤ/p` where the characteristic of this ring is p.
    type PrimeSubfield: PrimeField;

    const ZERO: Self;
    const ONE: Self;
    // etc.
```

The associated type of `PrimeCharacteristicRing` is of type `PrimeField`, which is defined as...

```
pub trait PrimeField:
    Field
    + ...
```

And then the `Field` trait has `Algebra<Self>` as a supertrait, which in turn has `PrimeCharacteristicRing` as a supertrait.

See [Lean] : support mutual dependencies (between items, between modules, inside traits/impl) for more context on the Hax side.

Equality Constraints and Trait Bounds on Associated Types

Let's look again at the `PrimeCharacteristicRing` definition:

```
pub trait PrimeCharacteristicRing:
    Sized
    + Default
    + Clone
    + Add<Output = Self>
    + AddAssign
    + Sub<Output = Self>
    + SubAssign
    + Neg<Output = Self>
    + Mul<Output = Self>
    + MulAssign
    + Sum
    + Product
    + Debug
{
    /// The field `ℤ/p` where the characteristic of this ring is p.
    type PrimeSubfield: PrimeField;

    const ZERO: Self;
    // etc.
```

Here we can see two features of associated types that are [Lean] Extend support for associated types by Hax:
equality constraints (e.g., `Add<Output = Self>`) and trait bounds (e.g., `type PrimeSubfield: PrimeField;`).

For both cases we have commented out the annotations.
We will be following the approach of relaxing these restrictions at the trait level while making sure that the logic at the implementation
level remains equivalent to the original implementation if not the same.

Unimplemented Traits and Functions

Some traits are not implemented on the Hax side are needed in order to perform a full extraction. So far, we have missed the

implementation of the `Iterator` trait, since both the `Sum` and `Product` supertraits of the `PrimeCharacteristicRing` trait depend on it.

Another case of unimplemented code are functions lacking implementations on the Hax side (e.g., `overflowing_add`).
However, for these cases we provide an implementation in `HaxPatch.lean` in order to not modify the Rust code.

Reproducibility

Another goal of this milestone is to achieve reproducibility of the verification efforts carried out. To this end, we have created a script, `extract.sh`, which performs the extraction of the Rust models with Hax and edits known errors that we know how to alleviate.

Connection with Specification Libraries

The bulk of the work for this milestone has been on learning the usage of the Hax tool, ironing out and reporting issues whenever necessary. An investigation into the specification counterpart of field arithmetic was conducted early on, but more development was necessary on the specification side. Additionally, there has not been time left to establish connections between the extracted code and the specification libraries.

It remains thus as immediate future work to link the extraction results with ComPoly and ArkLib.

Deliverables

Much of the work carried out in this milestone can be accounted for in intangible outcomes such as having insightful discussions and acquiring domain-knowledge on the Hax tools, the verification targets, and the ecosystem as a hole. However, there are tangible outcomes that have been a direct result of the work carried out here. These are the following.

Code

As mentioned before, the relevant code and framework that has been established (and will be organically growing) can be found in this repository: https://github.com/runtimeverification/Mersenne31

Issues

A number of issues have been filed during milestone 1:

- [Lean] `hax_lib::opaque` on trait impl generates invalid code (Opened as a result of a Zulip discussion)
- [Lean] Certain namings lead to invalid Lean
- [Lean] Trait constants used in trait functions are extracted with a fully quantified name
- [Lean] Invalid extraction for `const` implementations
- [Lean] Implemented items that use other associated items produce invalid Lean

Pull Requests

The following are pull requests related to the present ongoing work:

- https://github.com/cryspen/hax/pull/1887
- https://github.com/cryspen/hax/pull/1906

Next Steps

After having established the presented groundwork we have more clarity on what the next steps for this project will be. We list them in a coarse way:

- **Specification Connection**: Next up is to connect the extracted code to the specification libraries. The logic of our Rust models is nothing convoluted yet, so this should be a fine exercise to set up and learn what are the most immediate frictions.
- **Extract more convoluted functions**: Once we have a bridge between extracted code and the specification libraries, we will model Rust functions with nontrivial mathematics to truly test the proving boundaries of this approach.
- **Reproduce, iterate, and scale**: After a first version of an end-to-end verification pipeline is established we will make sure it is reproducible, iterate on more ambitious verification targets, and scale it at the best capacity possible.